

# DIVIDE AND CONQUER

- **Divide and Conquer** is a fundamental **paradigm** in the field of computer science and algorithm design, particularly in the context of Design and Analysis of Algorithms (**DAA**).
- This approach involves breaking down a complex problem into smaller, more manageable sub-problems, solving them independently, and then combining their solutions to obtain the solution to the original problem.
- The key idea is to divide the problem into smaller parts, conquer each part individually, and then merge the results.

The Divide and Conquer strategy typically follows these **three steps**:

## Divide:

- The original problem is divided into smaller, more manageable sub-problems.
- This step is crucial for simplifying the problem and making it easier to solve.
- The division process continues until the sub-problems become simple enough to be solved directly.

## Conquer:

- Each sub-problem is solved independently.
- This is often the recursive application of the same algorithm to each sub-problem.
- The base case of the recursion is the point where the sub-problems become simple enough to be solved directly, without further division.

**Combine:**

- The solutions to the sub-problems are combined to obtain the solution to the original problem.
- The combining step should be efficient and should not add significant complexity to the overall algorithm.

The **Divide and Conquer** paradigm is widely used in various algorithms and computational problems, such as **sorting algorithms (e.g., Merge Sort, QuickSort)**, **searching algorithms (e.g., Binary Search)**, and various dynamic programming solutions.

Here's a brief overview of how Divide and Conquer is applied in two common algorithms:

**❖ Merge Sort:**

- Divide: The array is divided into two halves.
- Conquer: Recursively sort each half.
- Combine: Merge the sorted halves to produce the final sorted array.

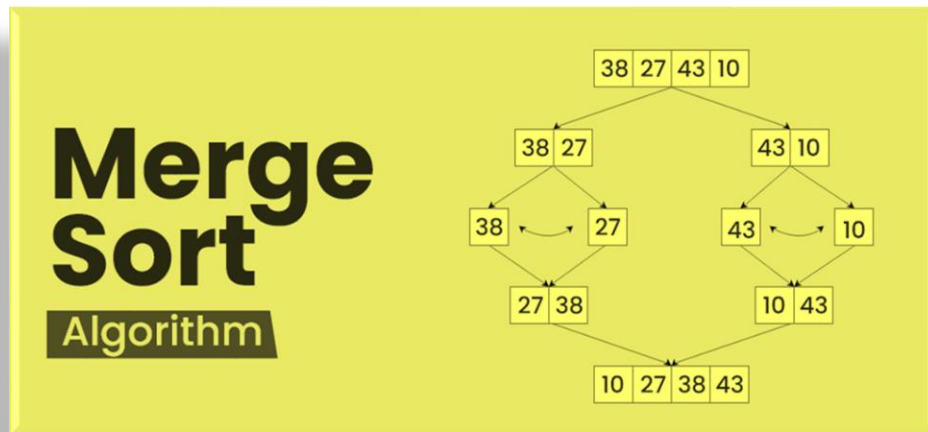
```

1. MERGE_SORT(arr, beg, end)
2. if beg < end
3. set mid = (beg + end)/2
4. MERGE_SORT(arr, beg, mid)
5. MERGE_SORT(arr, mid + 1, end)
6. MERGE (arr, beg, mid, end)
7. end of if
8. END MERGE_SORT

```

The important part of the merge sort is the **MERGE function**.

- ✓ This function performs the merging of two sorted sub-arrays that are **A[beg...mid]** and **A[mid+1...end]**, to build one sorted array **A[beg...end]**.
- ✓ So, the inputs of the MERGE function are **A[], beg, mid, and end**.



## How does Merge Sort work?

- Merge sort is a recursive algorithm that continuously splits the array in half until it cannot be further divided i.e., the array has only one element left (an array with one element is always sorted).
- Then the sorted subarrays are merged into one sorted array.

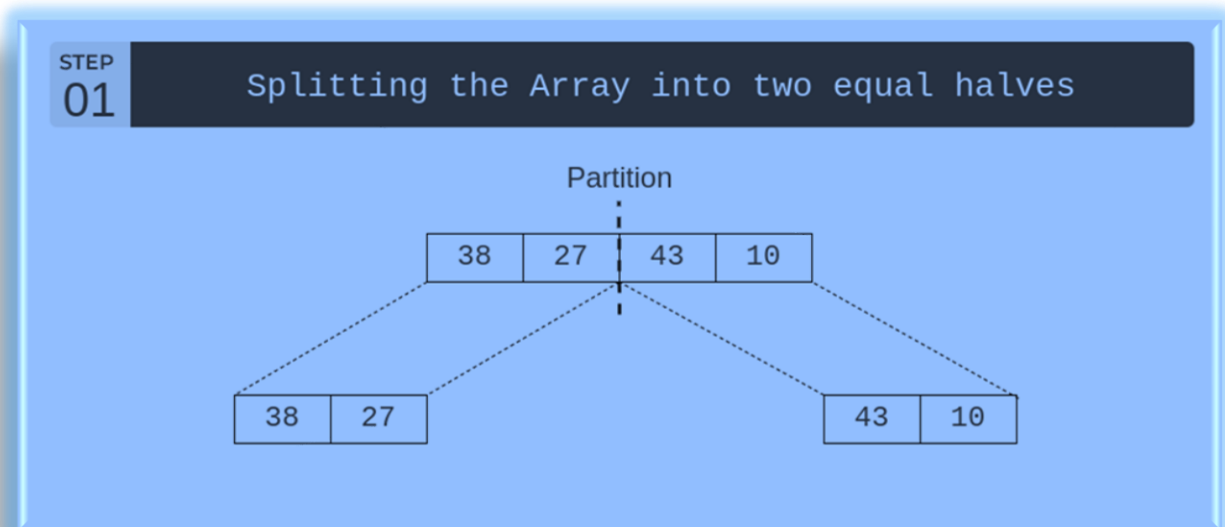
See the below **illustration** to understand the **working of merge sort**.

### Illustration:

©Topperworld

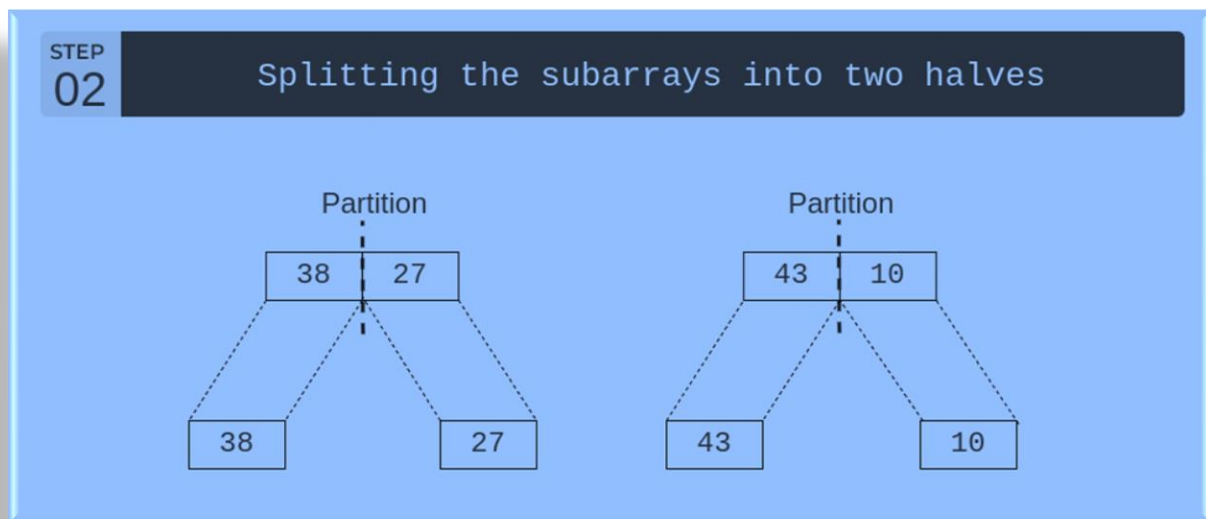
Lets consider an array `arr[] = {38, 27, 43, 10}`

Initially divide the array into **two** equal halves:

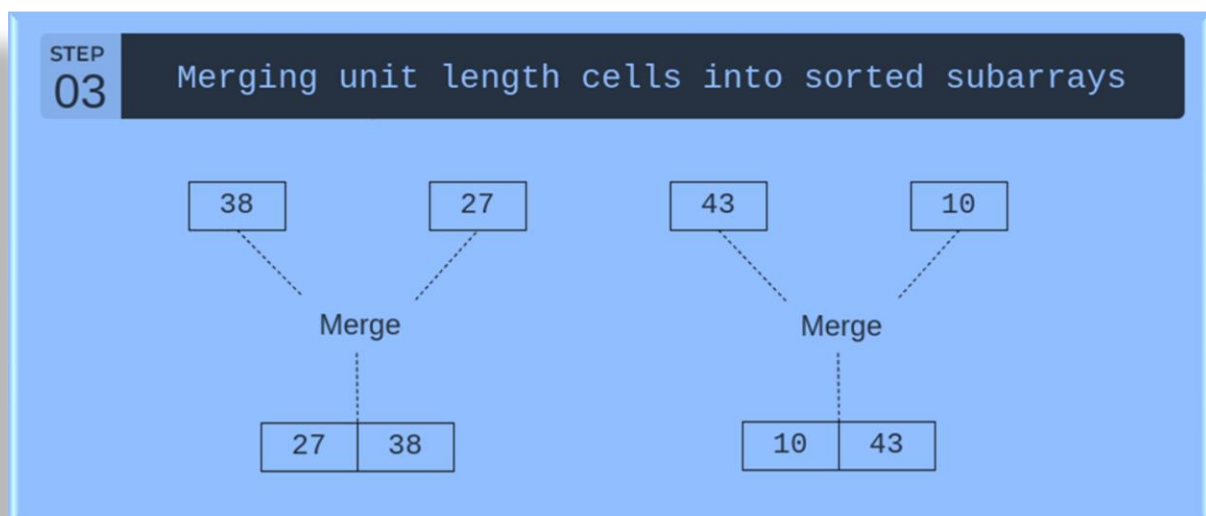


These subarrays are further divided into **two halves**.

Now they become array of unit length that can no longer be divided and array of unit length are always sorted.



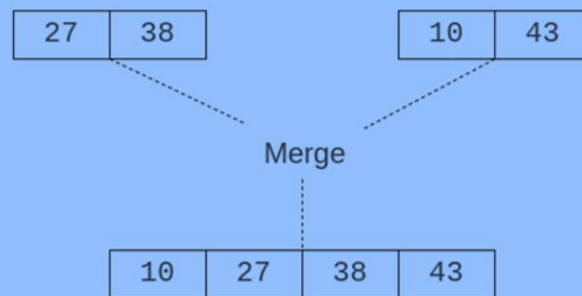
These sorted subarrays are **merged together**, and we get bigger sorted subarrays.



This merging process is continued until the sorted array is built from the smaller subarrays.

STEP  
04

## Merging sorted subarrays into the sorted array



The following diagram shows the complete merge sort process for an example array **{38, 27, 43, 10}**.

©Topperworld

```

include <iostream>
#include <vector>

void merge(std::vector<int>& arr, std::vector<int>& left, std::vector<int>& right) {
    int i = 0, j = 0, k = 0;
    while (i < left.size() && j < right.size()) {
        if (left[i] <= right[j]) {
            arr[k++] = left[i++];
        } else {
            arr[k++] = right[j++];
        }
    }
    while (i < left.size()) {
        arr[k++] = left[i++];
    }
    while (j < right.size()) {
        arr[k++] = right[j++];
    }
}

void merge_sort(std::vector<int>& arr) {
    if (arr.size() <= 1) {
        return;
    }
}
  
```

```
// Divide
int mid = arr.size() / 2;
std::vector<int> left(arr.begin(), arr.begin() + mid);
std::vector<int> right(arr.begin() + mid, arr.end());

// Recursively apply merge_sort to each half
merge_sort(left);
merge_sort(right);

// Combine (Merge)
merge(arr, left, right);
}

int main() {
    std::vector<int> arr = {38, 27, 43, 3, 9, 82, 10};

    std::cout << "Original Array: ";
    for (int num : arr) {
        std::cout << num << " ";
    }

    // Apply Merge Sort
    merge_sort(arr);

    std::cout << "\nSorted Array: ";
    for (int num : arr) {
        std::cout << num << " ";
    }

    return 0;
}
```

## Merge sort complexity

Now, let's see the time complexity of merge sort in best case, average case, and in worst case. We will also see the space complexity of the merge sort.

### 1. Time Complexity

- The best-case time complexity of Merge Sort is  $O(n \log n)$ .
- The average-case time complexity of Merge Sort is  $O(n \log n)$ .
- The worst-case time complexity of Merge Sort is  $O(n \log n)$ .

### 2. Space Complexity

- The Space Complexity is  $O(n)$

## Applications of Merge Sort:

### Sorting large datasets:

- Merge sort is particularly well-suited for sorting large datasets due to its guaranteed worst-case time complexity of  $O(n \log n)$ .

### External sorting:

- Merge sort is commonly used in external sorting, where the data to be sorted is too large to fit into memory.

### Custom sorting:

- Merge sort can be adapted to handle different input distributions, such as partially sorted, nearly sorted, or completely unsorted data.

## Advantages of Merge Sort:

### Stability:

- Merge sort is a stable sorting algorithm, which means it maintains the relative order of equal elements in the input array.

### Guaranteed worst-case performance:

- Merge sort has a worst-case time complexity of  $O(N \log N)$ , which means it performs well even on large datasets.

**Parallelizable:**

- Merge sort is a naturally parallelizable algorithm, which means it can be easily parallelized to take advantage of multiple processors or threads.

**Drawbacks of Merge Sort:****Space complexity:**

- Merge sort requires additional memory to store the merged sub-arrays during the sorting process.

**Not in-place:**

- Merge sort is not an in-place sorting algorithm, which means it requires additional memory to store the sorted data.
- This can be a disadvantage in applications where memory usage is a concern.

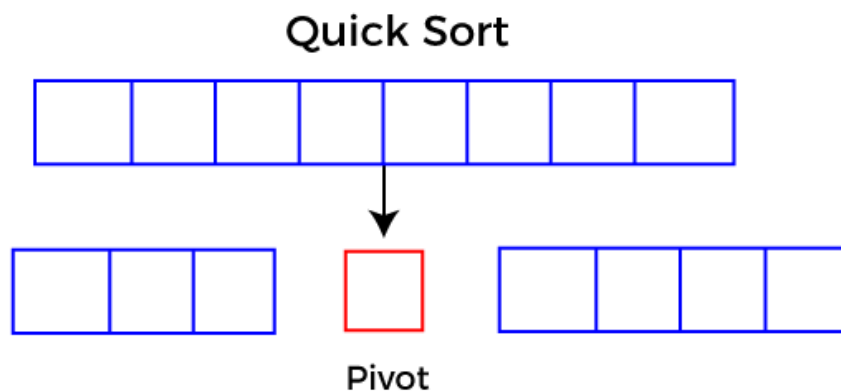
**Not always optimal for small datasets:**

- For small datasets, Merge sort has a higher time complexity than some other sorting algorithms, such as insertion sort



### ❖ QuickSort:

- Divide: Choose a pivot element and partition the array into two sub-arrays - elements less than the pivot and elements greater than the pivot.
- Conquer: Recursively sort the sub-arrays.
- Combine: No explicit combining step is needed; the sorting is done in place by rearranging the elements.



### Choosing the pivot

- Picking a good pivot is necessary for the fast implementation of quicksort.
- However, it is typical to determine a good pivot.

Some of the ways of choosing a pivot are as follows -

- ✓ Pivot can be random, i.e. select the random pivot from the given array.
- ✓ Pivot can either be the rightmost element of the leftmost element of the given array.
- ✓ Select median as the pivot element.

**Algorithm:**

```

QUICKSORT (array A, start, end)
{
  1 if (start < end)
  2 {
  3 p = partition(A, start, end)
  4 QUICKSORT (A, start, p - 1)
  5 QUICKSORT (A, p + 1, end)
  6 }
}

```

**Partition Algorithm:**

The partition algorithm rearranges the sub-arrays in a place.

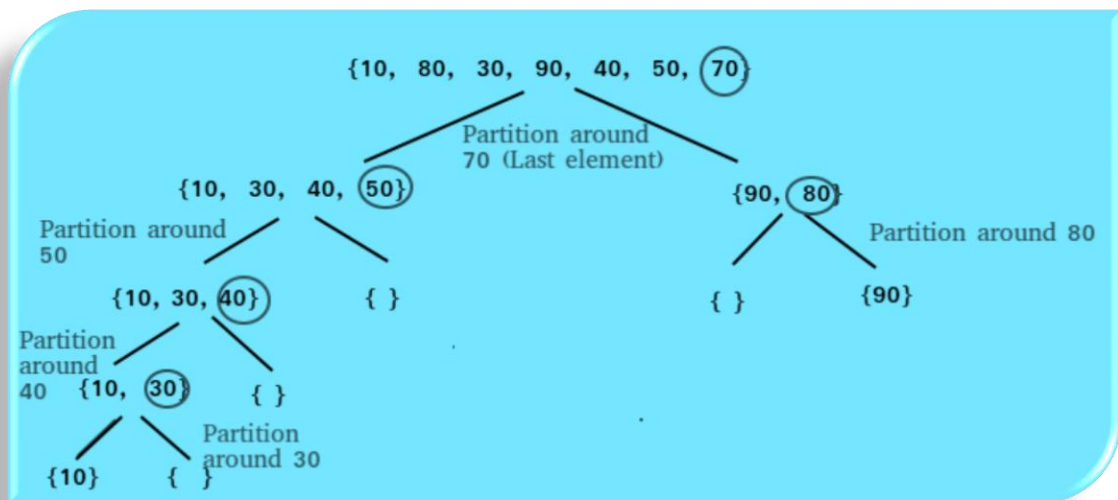
```

PARTITION (array A, start, end)
{
  1 pivot ? A[end]
  2 i ? start-1
  3 for j ? start to end -1 {
  4 do if (A[j] < pivot) {
  5 then i ? i + 1
  6 swap A[i] with A[j]
  7 }}
  8 swap A[i+1] with A[end]
  9 return i+1
}

```

## How does QuickSort work?

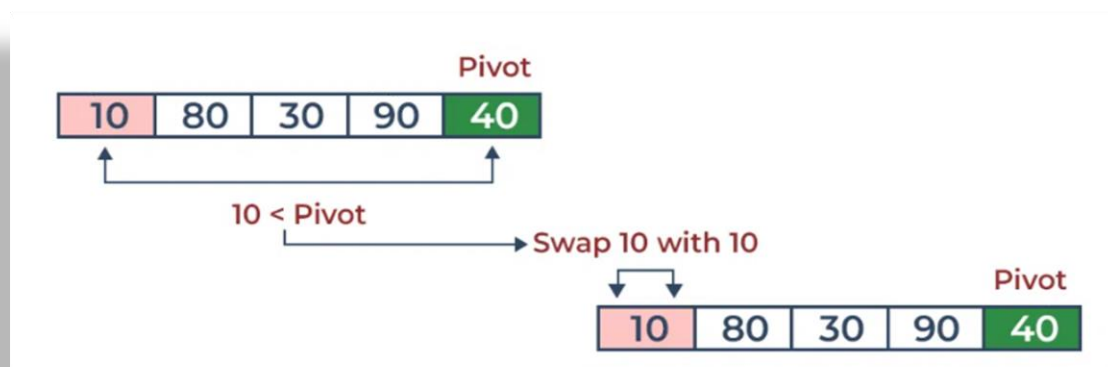
- ✓ The key process in quickSort is a partition().
- ✓ The target of partitions is to place the pivot (any element can be chosen to be a pivot) at its correct position in the sorted array and put all smaller elements to the left of the pivot, and all greater elements to the right of the pivot.
- ✓ Partition is done recursively on each side of the pivot after the pivot is placed in its correct position and this finally sorts the array.



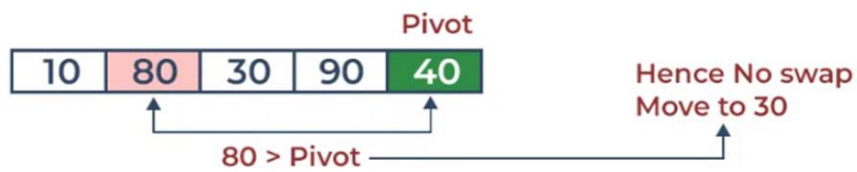
Let us understand the working of partition and the Quick Sort algorithm with the help of the following example:

Consider: `arr[] = {10, 80, 30, 90, 40}`.

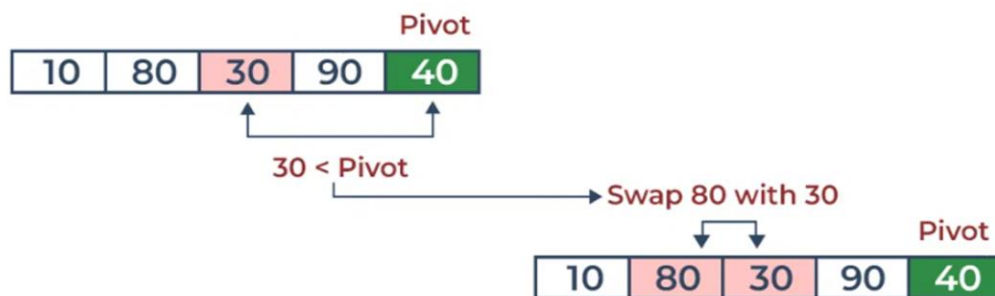
**Compare 10** with the **pivot** and as it is less than pivot arrange it accordingly.



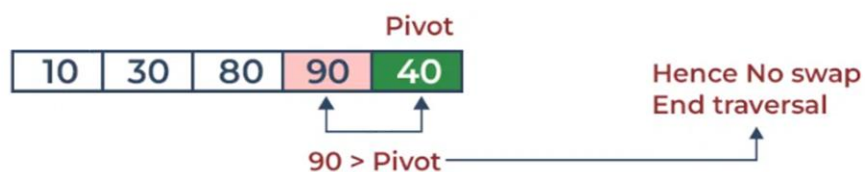
**Compare 80 with the pivot.** It is greater than pivot.



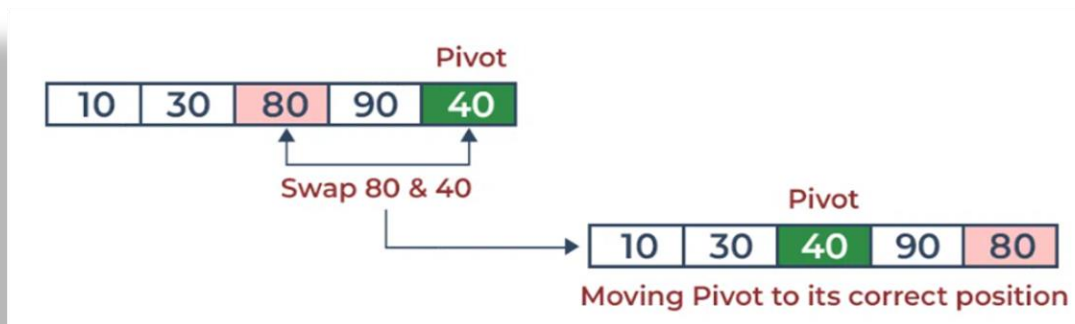
**Compare 30 with pivot.** It is less than pivot so arrange it accordingly.



**Compare 90 with the pivot.** It is greater than the pivot.



Arrange the pivot in its correct position.

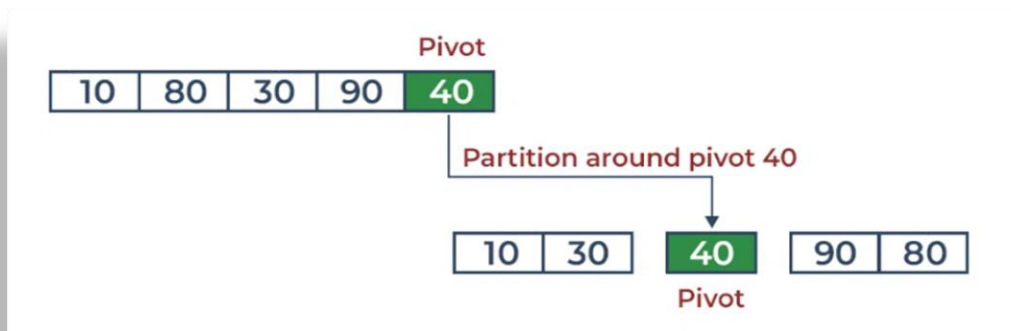


### illustration of Quicksort:

- ✓ As the partition process is done recursively, it keeps on putting the pivot in its actual position in the sorted array.
- ✓ Repeatedly putting pivots in their actual position makes the array sorted.

Follow the below images to understand how the recursive implementation of the partition algorithm helps to sort the array.

Initial partition on the main array:



Partitioning of the subarrays:



## Quicksort complexity

Now, let's see the time complexity of quicksort in best case, average case, and in worst case. We will also see the space complexity of quicksort.

### 1. Time Complexity

Best case =  $O(n \cdot \log n)$

Average case =  $O(n \cdot \log n)$

Worst case =  $O(n^2)$

2. Space Complexity =  $O(n \cdot \log n)$

## Implementation of Quick Sort:

```
#include <stdio.h>

int partition (int a[], int start, int end)
{
    int pivot = a[end]; // pivot element
    int i = (start - 1);

    for (int j = start; j <= end - 1; j++)
    {
        // If current element is smaller than the pivot
        if (a[j] < pivot)
        {
            i++; // increment index of smaller element
            int t = a[i];
            a[i] = a[j];
            a[j] = t;
        }
    }

    int t = a[i+1];
    a[i+1] = a[end];
```

```

a[end] = t;

return (i + 1);
}

/* function to implement quick sort */
void quick(int a[], int start, int end) /* a[] = array to be sorted, start =
Starting index, end = Ending index */
{
    if (start < end)
    {
        int p = partition(a, start, end); //p is the partitioning index

        quick(a, start, p - 1);

        quick(a, p + 1, end);
    }
}

/* function to print an array */
void printArr(int a[], int n)
{
    int i;
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
}

int main()
{
    int a[] = { 24, 9, 29, 14, 19, 27 };

    int n = sizeof(a) / sizeof(a[0]);

    printf("Before sorting array elements are - \n");

    printArr(a, n);

    quick(a, 0, n - 1);

    printf("\nAfter sorting array elements are - \n");

    printArr(a, n);
}

```

## OUTPUT:

```
Before sorting array elements are -  
24 9 29 14 19 27  
After sorting array elements are -  
9 14 19 24 27 29
```

### Advantages of Quick Sort:

- ✓ It is a divide-and-conquer algorithm that makes it easier to solve problems.
- ✓ It is efficient on large data sets.
- ✓ It has a low overhead, as it only requires a small amount of memory to function.

### Disadvantages of Quick Sort:

- ✓ It has a worst-case time complexity of  $O(N^2)$ , which occurs when the pivot is chosen poorly.
- ✓ It is not a good choice for small data sets.



## ❖ Binary Search:

- **Divide:** Compare the target value with the middle element of the sorted array.
  - **Conquer:** If the target is equal to the middle element, the search is complete. If the target is less than the middle element, search the left half; otherwise, search the right half.
  - **Combine:** No explicit combine step; the search space is reduced until the target is found or the sub-array becomes empty.
- ✓ **Binary Search** is defined as a searching algorithm used in a sorted array by repeatedly dividing the search interval in half.
- ✓ The idea of binary search is to use the information that the array is sorted and reduce the time complexity to  **$O(\log N)$** .

# Binary Search

|                                      |     |   |   |    |     |     |          |     |    |     |
|--------------------------------------|-----|---|---|----|-----|-----|----------|-----|----|-----|
|                                      | 0   | 1 | 2 | 3  | 4   | 5   | 6        | 7   | 8  | 9   |
| Search 23                            | 2   | 5 | 8 | 12 | 16  | 23  | 38       | 56  | 72 | 91  |
|                                      | L=0 | 1 | 2 | 3  | M=4 | 5   | 6        | 7   | 8  | H=9 |
| 23 > 16<br>take 2 <sup>nd</sup> half | 2   | 5 | 8 | 12 | 16  | 23  | 38       | 56  | 72 | 91  |
|                                      |     |   |   |    |     | L=5 | 6        | M=7 | 8  | H=9 |
| 23 < 56<br>take 1 <sup>st</sup> half | 2   | 5 | 8 | 12 | 16  | 23  | 38       | 56  | 72 | 91  |
|                                      |     |   |   |    |     |     | L=5, M=5 | H=6 |    |     |
| Found 23,<br>Return 5                | 2   | 5 | 8 | 12 | 16  | 23  | 38       | 56  | 72 | 91  |

Conditions for when to apply Binary Search in a Data Structure:

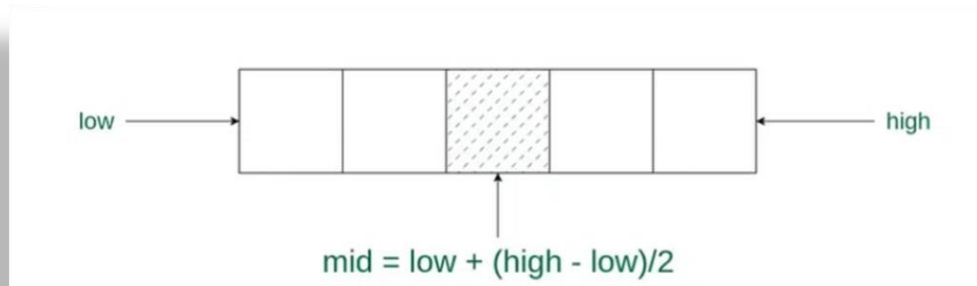
To apply **Binary Search algorithm**:

- ✓ The data structure must be sorted.
- ✓ Access to any element of the data structure takes constant time.

## Binary Search Algorithm:

In this algorithm,

Divide the search space into two halves by *finding the middle index "mid"*.



Compare the middle element of the search space with the key.

- If the key is found at **middle element**, the process is terminated.
- If the key is **not found at middle element**, choose which half will be used as the next search space.
- If the **key is smaller** than the middle element, then the left side is used for next search.
- If the **key is larger** than the middle element, then the right side is used for next search.
- This process is continued until the key is found or the total search space is exhausted.

## How does Binary Search work?

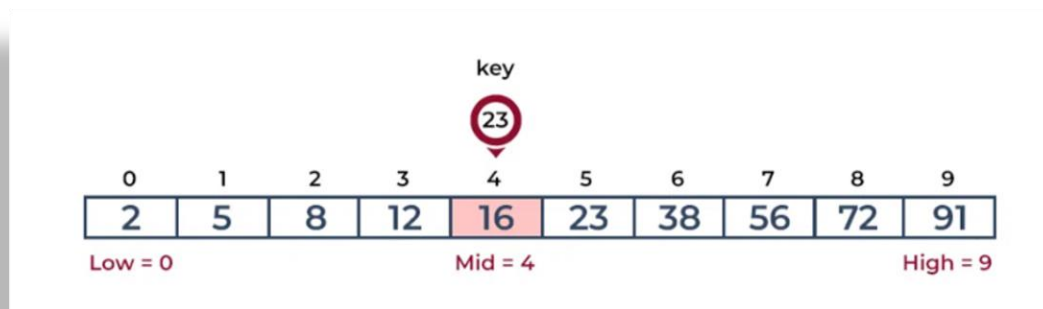
To understand the working of binary search, consider the following illustration:

Consider an **array** `arr[] = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91}`,

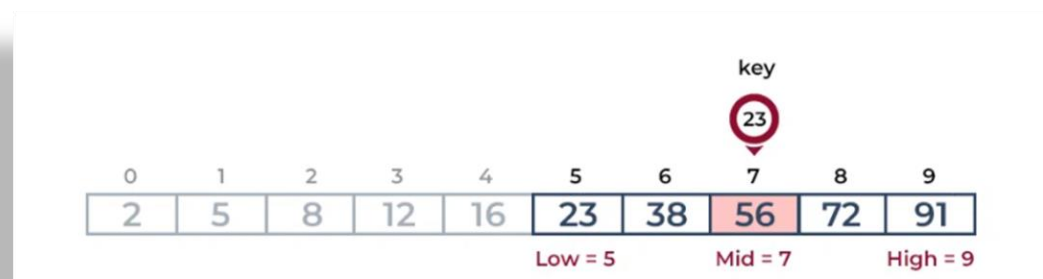
and the **target = 23**.

**First Step:**

- ✓ **Calculate the mid** and **compare the mid** element with the key.
- ✓ If the key is less than mid element, move to left and if it is greater than the mid then move search space to the right.
- ✓ Key (**i.e., 23**) is greater than current mid element (**i.e., 16**).
- ✓ The search space moves to the right.



Key is less than the current mid 56. The search space moves to the left.

**Second Step:**

- If the key matches the value of the mid element, the element is found and stop search.



## Implementation of Binary Search:

```
#include <iostream>

int binarySearch(int arr[], int size, int target) {
    int low = 0;
    int high = size - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] == target) {
            return mid; // Element found, return its index
        } else if (arr[mid] < target) {
            low = mid + 1; // Search in the right half
        } else {
            high = mid - 1; // Search in the left half
        }
    }

    return -1; // Element not found
}

int main() {
    int arr[] = {2, 5, 8, 12, 16, 23, 38, 42, 55, 67};
    int size = sizeof(arr) / sizeof(arr[0]);

    int target = 23;
    int result = binarySearch(arr, size, target);

    if (result != -1) {
        std::cout << "Element " << target << " found at index " << result << std::endl;
    } else {
```

```
std::cout << "Element " << target << " not found in the array." << std::endl;  
}  
return 0;  
}
```

## OUTPUT:

```
Element 23 found at index 5
```

## Complexity Analysis of Binary Search:

©Topperworld

### Time Complexity:

- Best Case:  $O(1)$
- Average Case:  $O(\log N)$
- Worst Case:  $O(\log N)$

**Auxiliary Space:**  $O(1)$ , If the recursive call stack is considered then the auxiliary space will be  $O(\log N)$ .

## Advantages of Binary Search:

- ✓ Binary search is faster than linear search, especially for large arrays.
- ✓ More efficient than other searching algorithms with a similar time complexity, such as interpolation search or exponential search.
- ✓ Binary search is well-suited for searching large datasets that are stored in external memory, such as on a hard drive or in the cloud.

## Drawbacks of Binary Search:

- ✓ The array should be sorted.

- ✓ Binary search requires that the data structure being searched be stored in contiguous memory locations.
- ✓ Binary search requires that the elements of the array be comparable, meaning that they must be able to be ordered.

### Applications of Binary Search:

- ✓ Binary search can be used as a building block for more complex algorithms used in machine learning, such as algorithms for training neural networks or finding the optimal hyperparameters for a model.
- ✓ It can be used for searching in computer graphics such as algorithms for ray tracing or texture mapping.
- ✓ It can be used for searching a database.

### ❖ Strassen's Matrix Multiplication:

- **Divide:** Split the input matrices into four equal-sized sub-matrices.
- **Conquer:** Recursively multiply the sub-matrices using seven multiplications (instead of the standard eight).
- **Combine:** Combine the results to form the final product.

### Algorithm of Matrix Chain – Multiplication:

Algorithm: Matrix-Multiplication (X, Y, Z)

for i = 1 to p do

for j = 1 to r do

Z[i,j] := 0

for k = 1 to q do

Z[i,j] := Z[i,j] + X[i,k] × Y[k,j]

- **Strassen's Matrix multiplication** can be performed only on square matrices where  $n$  is a power of 2.
- Order of both of the matrices are  $n \times n$ .
- Divide **X**, **Y** and **Z** into four  $(n/2) \times (n/2)$  matrices as represented below:

$$Z = \begin{bmatrix} I & J \\ K & L \end{bmatrix} \quad X = \begin{bmatrix} A & B \\ C & D \end{bmatrix} \quad \text{and} \quad Y = \begin{bmatrix} E & F \\ G & H \end{bmatrix}$$

Using Strassen's Algorithm compute the following –

$$\mathbf{M1:} = (A+C) \times (E+F)$$

$$\mathbf{M2:} = (B+D) \times (G+H)$$

$$\mathbf{M3:} = (A-D) \times (E+H)$$

©Topperworld

$$\mathbf{M4:} = A \times (F-H)$$

$$\mathbf{M5:} = (C+D) \times (E)$$

$$\mathbf{M6:} = (A+B) \times (H)$$

$$\mathbf{M7:} = D \times (G-E)$$

Then,

$$\mathbf{I:} = M2 + M3 - M6 - M7$$

$$\mathbf{J:} = M4 + M6$$

$$\mathbf{K:} = M5 + M7$$

$$\mathbf{L:} = M1 - M3 - M4 - M5$$

**Analysis:**

$$T(n) = \begin{cases} c & \text{if } n = 1 \\ 7 \times T(\frac{n}{2}) + d \times n^2 & \text{otherwise} \end{cases} \text{ where } c \text{ and } d \text{ are constants}$$

Using this recurrence relation, we get

$$T(n) = O(n^{\log_2 7})$$

Hence, the complexity of Strassen's matrix multiplication algorithm is :

$$O(n^{\log_2 7}).$$

**Implementation of Strassen's matrix multiplication:**

```
#include<stdio.h>

int main(){
    int z[2][2];
    int i, j;
    int m1, m2, m3, m4, m5, m6, m7;
    int x[2][2] = {{12, 34},{22, 10}};
    int y[2][2] = {{3, 4}, {2, 1}};
    printf("The first matrix is: ");
    for(i = 0; i < 2; i++) {
        printf("\n");
        for(j = 0; j < 2; j++)
            printf("%d\t", x[i][j]);
    }
    printf("\nThe second matrix is: ");
    for(i = 0; i < 2; i++) {
        printf("\n");
```



```

for(j = 0; j < 2; j++)
    printf("%d\t", y[i][j]);
}

m1= (x[0][0] + x[1][1]) * (y[0][0] + y[1][1]);
m2= (x[1][0] + x[1][1]) * y[0][0];
m3= x[0][0] * (y[0][1] - y[1][1]);
m4= x[1][1] * (y[1][0] - y[0][0]);
m5= (x[0][0] + x[0][1]) * y[1][1];
m6= (x[1][0] - x[0][0]) * (y[0][0]+y[0][1]);
m7= (x[0][1] - x[1][1]) * (y[1][0]+y[1][1]);

z[0][0] = m1 + m4- m5 + m7;
z[0][1] = m3 + m5;
z[1][0] = m2 + m4;
z[1][1] = m1 - m2 + m3 + m6;

printf("\nProduct achieved using Strassen's algorithm: ");
for(i = 0; i < 2 ; i++) {
    printf("\n");
    for(j = 0; j < 2; j++)
        printf("%d\t", z[i][j]);
}

return 0;
}

```

**OUTPUT:**

```

The first matrix is:
12    34
22    10
The second matrix is:
3     4
2     1
Product achieved using Strassen's algorithm:
174    146
110    88

```

- **Time complexity** of Strassen's Matrix Multiplication is:

$$T(n) = O(n^{\log_2 7}) \approx O(n^{2.81})$$

- This time complexity is better than the naive matrix multiplication algorithm which is  **$O(n^3)$**
- **Space complexity** of Strassen's Matrix Multiplication is:  **$S(n)=O(\log n)$**

### ❖ Closest Pair of Points:

- **Divide:** Divide the set of points into two halves based on their x-coordinates.
- **Conquer:** Recursively find the closest pair of points in each half.
- **Combine:** Check for the closest pair across the dividing line.

The Divide and Conquer paradigm is **powerful** because it often leads to efficient algorithms with **logarithmic or linearithmic time complexity**, making it a fundamental concept in algorithm **design and analysis**.

### Algorithm:

- 1) We sort all points according to x coordinates.
- 2) Divide all points in two halves.
- 3) Recursively find the smallest distances in both subarrays.
- 4) Take the minimum of two smallest distances. Let the minimum be d.
- 5) Create an array strip[] that stores all points which are at most d distance away from the middle line dividing the two sets.
- 6) Find the smallest distance in strip[].
- 7) Return the minimum of d and the smallest distance calculated in above step 6.

**Certainly!** Let's walk through an example of the Closest Pair of Points problem using the Divide and Conquer algorithm.

Consider the following set of points in a 2D plane:

$P = \{(1,2), (4,6), (7,8), (9,5), (12,2), (15,14), (17,6), (19,9)\}$

**Step 1:**

- ✓ Sort Points Based on x-coordinates.
- ✓ Sort the points based on their x-coordinates:

$P = \{(1,2), (4,6), (7,8), (9,5), (12,2), (15,14), (17,6), (19,9)\}$

**Step 2:**

- ✓ Divide the Set into Two Halves
- ✓ Divide the sorted set into two equal halves along the vertical line passing through the median x-coordinate:

$P_{\text{left}} = \{(1,2), (4,6), (7,8), (9,5)\}$

$P_{\text{right}} = \{(12,2), (15,14), (17,6), (19,9)\}$

**Step 3:**

- ✓ Recursively Find the Closest Pair in Each Half
- ✓ Recursively find the closest pair in each half.
- ✓ We'll apply the algorithm to both.

**Step 4:**

- ✓ Merge the Two Halves
- ✓ Determine the minimum distance between pairs that have one point in each half.
- ✓ This involves checking a strip of points around the median line.

**Step 5:**

- ✓ Return the Closest Pair
- ✓ Return the pair with the smallest distance found in the previous steps.

**Visualization:**

- For the sake of simplicity, let's assume that we have found the closest pair in each half and are now considering the strip of points around the median line.

Consider the **strip**:

**Strip** = {(7,8),(9,5),(12,2),(15,14)}

- ✓ We need to find the closest pair within this strip.
- ✓ This can be done efficiently by comparing each point to the next 7 points in the strip (as the maximum possible distance is 7).

After checking all possible pairs in the strip, we find that the closest pair in the strip is (9,5) and (12,2) with a distance of:

$$\sqrt{(12 - 9)^2 + (2 - 5)^2} = \sqrt{9 + 9} = \sqrt{18}.$$

- ✓ Now, we compare this distance with the distances found in the left and right halves.
- ✓ If the distance in the strip is smaller, it becomes the final result.
- ✓ Otherwise, we return the minimum distance found in the left or right halves.
- ✓ In this example, let's assume that the closest pair in the strip has the smallest distance.

### Final Result:

The closest pair of points in the entire set  $P$  is  $(9, 5)$  and  $(12, 2)$  with a distance of  $\sqrt{18}$ .

### Time Complexity:

- The time complexity of the Closest Pair of Points algorithm is  $O(n \log n)$ , where  $n$  is the number of points.

### Space Complexity:

- The space complexity of the Closest Pair of Points algorithm is  $O(n)$ .

## Application of Divide and Conquer:

### Sorting Algorithms:

- **Merge Sort:**

Efficiently sorts arrays or lists by dividing them into halves, sorting each half, and then merging the sorted halves.

- **QuickSort:**

Divides an array into partitions, sorts each partition, and combines them to achieve sorting.

### Searching Algorithms:

- **Binary Search:**

Finds an element in a sorted array by repeatedly dividing the search space in half.

- **Strassen's Algorithm:**

Multiplies two matrices using a recursive divide and conquer approach, reducing the number of multiplications compared to the standard algorithm.

### Closest Pair of Points:

- **Closest Pair Problem:**

Finds the pair of points with the smallest Euclidean distance among a set of points in a 2D plane.

### Advantage of Divide and Conquer:

- **Efficiency:** Offers efficient solutions with good time complexity.
- **Parallelization:** Allows parallel processing for improved performance.
- **Simplicity:** Simplifies complex problems by breaking them into manageable subproblems.
- **Reusability:** Promotes reusability of solutions to subproblems.
- **Versatility:** Applicable to a wide range of problems.
- **Optimization:** Enables optimization by avoiding redundant computations.

### Disadvantage of Divide and Conquer:

- **Overhead:** Introduces additional overhead due to recursive calls and combining solutions.
- **Not Always Optimal:** May not always lead to the most optimal solution.
- **Difficulty in Identifying Subproblems:** Identifying the right subproblems can be challenging.
- **Memory Consumption:** Recursive algorithms may consume significant memory.
- **Sequential Dependency:** Not suitable for problems with inherent sequential dependencies.